architecture.py — technical

`src/moe_pruning/model/architecture.py · 219 lines · moe_pruning.model.architecture`

Purpose

Abstract interface and data containers. Concrete behaviour lives in plugin adapters; default implementations here assume one module per expert addressed by a state-dict prefix.

Data and structure

- `MoEForwardRecord` shapes: residual/moe_input/moe_output `[B, S, D]`, expert_index/gate `[B, S, K]`, expert_output `[B, S, K, D]`; index -1 marks 'no expert' (capacity drop).

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class MoELayerInfo`

lines 20-38 · dataclass

Static description of one sparse MoE layer.

Field	Type	Default
`layer_id`	str	
`stack`	str	
`position`	int	
`module_path`	str	
`router_path`	str	
`experts_path`	str	
`num_experts`	int	
`top_k`	int	
`hidden_dim`	int	
`expert_input_dim`	int	
`expert_intermediate_dim`	int	
`expert_output_dim`	int	
`expert_capacity`	int | None	

`MoELayerInfo.to_dict`

```
def to_dict(self) -> dict
```

lines 37-38

Convert to a dictionary (stored in manifests and reports).

`class MoEForwardRecord`

lines 42-56 · dataclass

Everything observed in one forward call of one sparse MoE layer.

Field	Type	Default
`layer_id`	str	
`residual`	torch.Tensor	
`moe_input`	torch.Tensor	
`moe_output`	torch.Tensor	
`expert_index`	torch.Tensor	
`gate`	torch.Tensor	
`expert_output`	torch.Tensor	

class MoEAdapter

lines 62-194 · bases ABC

Model-specific access to sparse MoE layers.

MoEAdapter.discover_layers

```
def discover_layers(self, model: nn.Module) -> list[MoELayerInfo]
```

lines 70-71

Find all sparse MoE layers by inspecting the loaded model.

MoEAdapter.get_moe_module

```
def get_moe_module(self, model: nn.Module, layer: MoELayerInfo) -> nn.Module
```

lines 74-74

The sparse MoE module of a layer.

MoEAdapter.get_router

```
def get_router(self, model: nn.Module, layer: MoELayerInfo) -> nn.Module
```

lines 77-77

The router module of a layer.

MoEAdapter.get_expert

```
def get_expert(self, model: nn.Module, layer: MoELayerInfo, expert_id: int) -> nn.Module
```

lines 80-80

Module computing one expert (original, view or replacement).

MoEAdapter.set_expert

```
def set_expert(self, model: nn.Module, layer: MoELayerInfo, expert_id: int, module: nn.Module) ->
nn.Module
```

lines 83-86

Install `module` in the expert slot and return the previous module.

MoEAdapter.build_original_expert

```
def build_original_expert(self, model: nn.Module, layer: MoELayerInfo) -> nn.Module
```

lines 89-90

New (uninitialised) expert module of the model's original expert class.

MoEAdapter.expert_state_prefix

```
def expert_state_prefix(self, layer: MoELayerInfo, expert_id: int) -> str
```

lines 93-94

State-dict key prefix of an expert slot (ends with a dot).

MoEAdapter.route

```
def route(self, model: nn.Module, layer: MoELayerInfo, moe_input: torch.Tensor, apply_mask: bool=True)
-> tuple[torch.Tensor, torch.Tensor]
```

lines 97-104

Return (expert_index [B,S,K], gate [B,S,K]) exactly as used by the layer.

MoEAdapter.set_routing_mask

```
def set_routing_mask(self, model: nn.Module, layer: MoELayerInfo, expert_ids: list[int]) -> None
```

lines 107-111

Make `expert_ids` unreachable: their tokens go to the best remaining expert.

`MoEAdapter.get_routing_mask`

```
def get_routing_mask(self, model: nn.Module, layer: MoELayerInfo) -> list[int]
```

lines 114-114

Experts currently made unreachable by the reroute mask.

`MoEAdapter.run_moe_layer`

```
def run_moe_layer(self, model: nn.Module, layer: MoELayerInfo, moe_input: torch.Tensor) -> torch.Tensor
```

lines 117-118

Execute only the sparse MoE module on `moe_input` ([B,S,D]).

`MoEAdapter.instrument`

```
def instrument(self, model: nn.Module, layers: list[MoELayerInfo], callback: RecordCallback) ->
Iterator[None]
```

lines 122-125

Call `callback` with a `MoEForwardRecord` per layer forward.

`MoEAdapter.describe`

```
def describe(self, model: nn.Module) -> dict
```

lines 128-129

Machine-readable architecture report.

`MoEAdapter.replace_experts_in_layer`

```
def replace_experts_in_layer(self, model: nn.Module, layer: MoELayerInfo, modules: dict[int, nn.Module])
-> int
```

lines 134-147

Install `modules` in their expert slots; return the number of removed parameters.

1. Default: for each `(expert_id, module)` in id order call `set_expert`, add the old module's parameter count, delete the old module; return the total removed parameters.

`MoEAdapter.removed_weight_violations`

```
def removed_weight_violations(self, state_shapes: dict[str, tuple[int, ...]], layer: MoELayerInfo,
removed: list[int]) -> list[str]
```

lines 149-161

Names of stored tensors that still hold weights of removed experts.

1. Default: for each removed expert, every stored key under its `expert_state_prefix` other than `prefix + 'bias'` is a violation.

`MoEAdapter.load_original_expert`

```
def load_original_expert(self, reader, model: nn.Module, layer: MoELayerInfo, expert_id: int) ->
nn.Module
```

lines 163-174 · raises KeyError

Rebuild one original expert from the original checkpoint (`reader`: CheckpointReader).

1. Default: collect reader tensors under the expert's prefix (prefix stripped), build a fresh original expert via `build_original_expert`, `load_state_dict(strict=True)`; missing tensors -> `KeyError`.

`MoEAdapter.validate_loaded`

```
def validate_loaded(self, model: nn.Module, layers: list[MoELayerInfo]) -> None
```

lines 176-177

Called by the checkpoint loader after weights are loaded; raise on inconsistency.

`MoEAdapter.get_experts`

```
def get_experts(self, model: nn.Module, layer: MoELayerInfo) -> dict[int, nn.Module]
```

lines 182-183

All expert modules of a layer by id.

`MoEAdapter.hidden_dims`

```
def hidden_dims(self, model: nn.Module) -> dict[str, dict[str, int]]
```

lines 185-194

Hidden and expert dimensions of every MoE layer.

`select_layers`

```
def select_layers(layers: list[MoELayerInfo], spec: str) -> list[MoELayerInfo]
```

lines 197-212 · raises ValueError

Resolve PRUNE_LAYERS: `first`, `all` or comma-separated layer ids.

1. `first` -> first discovered layer; `all` -> all; otherwise comma-separated ids, unknown ids -> `ValueError` listing the valid ids; result keeps model order and drops duplicates.

`get_adapter`

```
def get_adapter(model: nn.Module) -> MoEAdapter
```

lines 215-219

Adapter of the model's plugin (looked up by `config.model_type`).

1. Look up the plugin by `model.config.model_type` and instantiate its adapter.

Errors raised

Function	Exception	Message (f-string fields in braces)
`select_layers`	ValueError	The model has no sparse MoE layers
`select_layers`	ValueError	Unknown MoE layer id(s) {unknown}. Available: {sorted(by_id)}
`load_original_expert`	KeyError	No tensors with prefix {prefix} in the original checkpoint